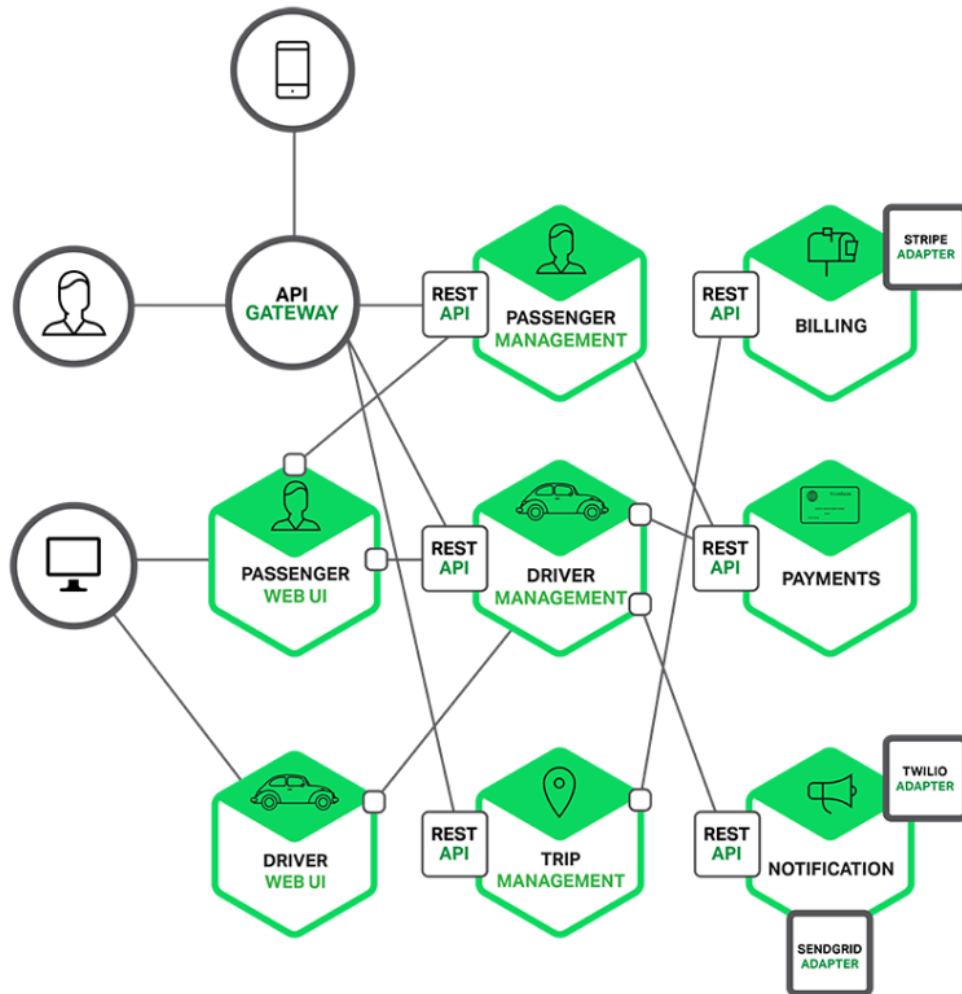


WEEK_3

API Gateway Service

MSA에서 클라이언트 요청을 받아 각 마이크로서비스로 라우팅하고, 인증·보안·로그·요청 합치기 같은 공통 기능을 중앙에서 처리하는 서비스

- 사용자는 Gateway한테만 요청하고, Gateway가 알아서 필요한 서비스로 요청을 보내주고 응답을 다시 돌려줌



API Gateway의 주요 기능

- 인증 및 권한 부여
 - 토큰 검증, 사용자 권한 체크를 Gateway 단에서 처리 → 마이크로서비스는 비즈니스 로직에 집중
- 서비스 검색 통합
 - 동적으로 생성/종료되는 서비스 인스턴스들의 위치를 자동으로 찾아서 라우팅
- 응답 캐싱
 - 자주 요청되는 응답을 미리 저장해두고 재사용 → 서비스 호출 횟수와 응답 속도 최적화
- 정책, 회로 차단기 및 QoS 다시 시도
 - 장애 발생 시 요청 차단 · 대체 응답 제공

- 일시적 실패 시 자동 재시도 → 전체 서비스 안정성 확보
- 속도 제한
 - 특정 IP/사용자가 일정 시간 동안 보낼 수 있는 요청 수 제한 → 서버 과부하 · DDoS 방어
- 부하 분산
 - 같은 서비스의 여러 인스턴스에 요청을 균등하게 분배
- 로깅, 추적, 상관 관계
 - 요청/응답 흐름 추적, 분산 트랜잭션 추적 ID 관리
- 헤더, 쿼리 문자열 및 청구 변환
 - 요청 · 응답 포맷 변경, 헤더 추가/삭제, 쿼리 파라미터 변환 등
- IP 허용 목록에 추가
 - 특정 IP만 접근 허용 → 민감 API 보호

Netflix Ribbon

- 현재는 공식 유지보수를 종료 (deprecated)

MSA에서 클라이언트 사이드 로드밸런싱 용으로 사용됨. 현재는 Spring Cloud LoadBalancer, 서비스 디스커버리 + Gateway 조합으로 대체

- **과거:** "Ribbon + Eureka" 조합이 표준
- **현재:** "Spring Cloud LoadBalancer + Eureka(or Consul)" 또는 "Kubernetes + Service Mesh"

Spring Cloud에서의 MSA 간 통신 (흔히 사용했던 두 가지 방법)

1. RestTemplate - Spring에서 제공하는 동기식 HTTP 요청/응답 템플릿
 - GET, POST, PUT, DELETE 등 HTTP 메서드를 직접 사용
 - URL, 헤더, 바디 등을 개발자가 직접 설정
 - 간단한 서비스 호출에는 좋지만, 서비스가 많아지면 코드가 장황해지고 유지보수가 어려움
2. Feign Client - 선언적(Declarative) HTTP 클라이언트로, Spring Cloud Netflix에서 제공
 - 인터페이스를 정의하고, 어노테이션으로 호출할 API를 지정
 - URL/헤더/파라미터 매핑이 간단
 - Load Balancing(Eureka)과 결합하면 서비스 이름으로 바로 호출 가능
 - 유지보수가 쉽고, 코드가 간결

Netflix Zuul

- 현재는 공식 유지보수를 종료 (deprecated)

Netflix에서 개발한 API Gateway 서버로, Spring Cloud와 연동해 마이크로서비스 아키텍처에서 사용됨



서비스 흐름 예시

1. 클라이언트 → Zuul Gateway
클라이언트는 각 마이크로서비스의 위치를 알 필요 없이 Zuul에만 요청을 보냄
2. Zuul이 요청 라우팅 → First Service 또는 Second Service
Zuul이 요청을 적절한 서비스로 전달하고, 각 서비스는 비즈니스 로직을 수행
3. 각 서비스 처리 후 응답 → Zuul → 클라이언트 전달
처리 결과를 Zuul이 다시 받아 클라이언트에 반환

Zuul의 역할

- 클라이언트는 마이크로서비스 위치를 알 필요 없이 Zuul만 호출하면 됨
- 인증/권한 체크, 로깅, 요청·응답 변환 등 공통 기능을 중앙에서 처리
- 여러 인스턴스의 서비스 간 로드 밸런싱 가능

기타 특징

- Pre, Route, Post 필터를 통해 요청 전/중/후 처리 가능
- Spring Cloud Netflix와 통합하여 Eureka 서비스 디스커버리와 연동 가능

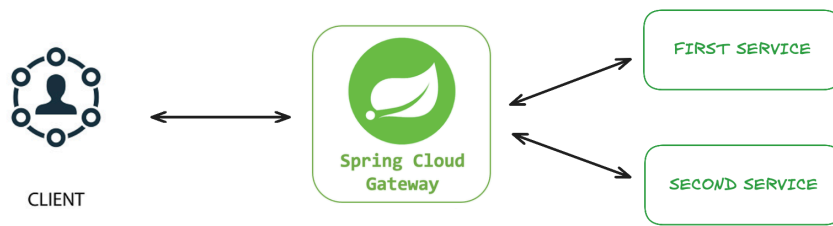
현재 Netflix Zuul은 사용되지 않으며, Spring Cloud Gateway 등 다른 API Gateway로 대체됨

Spring Cloud Gateway

마이크로서비스(MSA) 환경에서 API Gateway 역할을 하며, 내부적으로 WebFlux 또는 Webmvc 기반으로 구현 가능

- 클라이언트 요청을 적절한 서비스로 라우팅(Routing)
- 인증/권한 체크, 로깅, 요청·응답 변환 등 공통 기능 처리
- 로드 밸런싱, 속도 제한, 회로 차단기(Circuit Breaker) 등 제공

→ Zuul 1.x의 현대적 대체제이며 Spring WebFlux 기반으로 비동기/리액티브 처리가 가능



WebMvc vs. WebFlux

Spring 기반 웹 애플리케이션을 개발할 때 선택할 수 있는 두 가지 웹 모듈

구분	Spring Webmvc	Spring WebFlux
방식	동기식(Synchronous) 처리	비동기/리액티브(Asynchronous/Reactive) 처리
모델	Servlet API 기반, 전통적인 스레드 블로킹 방식	Netty, Undertow 등 이벤트 루프 기반, 논블로킹 처리
장점	구현과 디버깅이 단순, 기존 코드와 호환성 높음	높은 동시성, 적은 스레드로 많은 요청 처리 가능, MSA·Reactive 서비스에 적합
사용 사례	전통적인 REST API, CRUD 중심 애플리케이션	실시간 데이터, WebSocket, 스트리밍, MSA API Gateway
서비스	사내 ERP 시스템, 고객 관리(CRM), 상품 정보 조회 API	채팅 서버, 실시간 주식 시세 API, Spring Cloud Gateway

동기(Synchronous)

- 요청 → 처리 → 응답 순서대로 진행
- 하나의 작업이 끝나야 다음 작업이 시작됨

비동기(Asynchronous)

- 요청 → 즉시 반환 또는 다른 작업 수행, 처리 완료 시 콜백/알림으로 응답
- 여러 작업이 동시에 진행될 수 있음

Filter

웹 애플리케이션에서 요청(Request)과 응답(Response)을 가로채 처리할 수 있는 컴포넌트

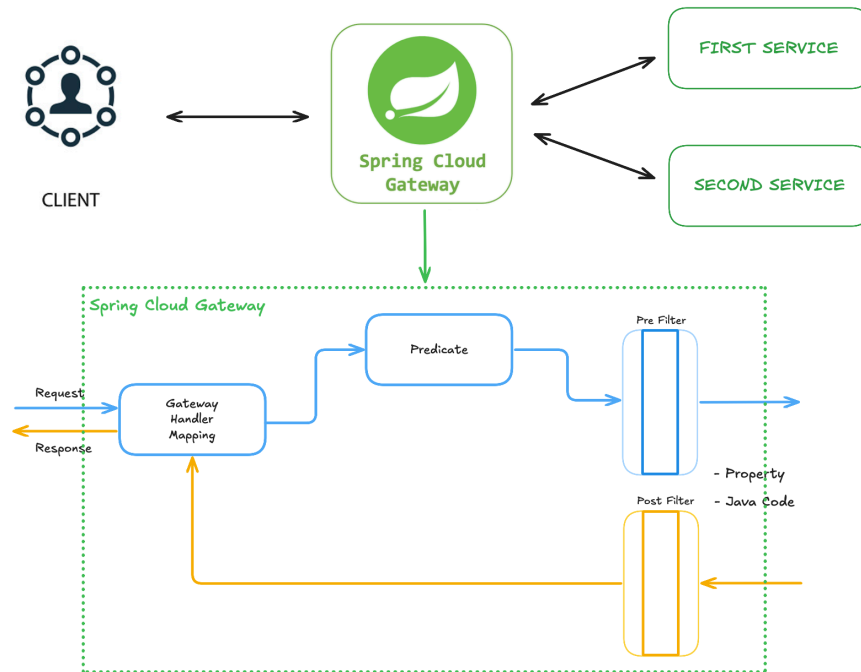
→ 요청이 실제 컨트롤러(서비스)로 전달되기 전이나, 응답이 클라이언트로 가기 전에 공통 작업 수행

1. 요청 전 처리(Pre-Filter)

- 인증/권한 확인
- 로깅 및 요청 정보 기록
- 헤더나 파라미터 조작

2. 응답 후 처리(Post-Filter)

- 응답 변환
- 응답 로깅
- 에러 처리, 회로 차단기 동작



Spring Cloud Gateway 실행 구조

- Predicate: 조건 검사 (이 요청을 어느 라우트로 보낼지 판별)
- Handler/Mapping: 라우팅할 서비스 결정
- Filter: 요청/응답을 변환, 로깅, 인증 처리

Custom Filter

웹 애플리케이션의 요청(Request)과 응답(Response) 처리 과정에서 개발자가 **직접 정의한 로직**을 추가하여 특정 작업을 수행하는 컴포넌트

기존 프레임워크나 라이브러리에서 제공하지 않는 고유하고 특화된 기능을 구현하거나, 애플리케이션에 특화된 방식으로 요청 및 응답을 제어할 때 사용됨

Ex)

- 특정 API 요청에만 적용되는 복잡한 토큰 검증 로직 구현
- 애플리케이션 고유의 비즈니스 규칙에 따른 요청 데이터 유효성 검사
- 특정 사용자 그룹에 대한 접근 제어 로직 커스터마이징

Custom Filter 정의

```
@Component
@Slf4j
public class CustomFilter extends AbstractGatewayFilterFactory<CustomFilter.Config> {

    public CustomFilter() {
        super(Config.class);
    }
}
```

```

@Override
public GatewayFilter apply(Config config) {
    // Custom Pre Filter
    return (exchange, chain) -> {
        ServerHttpRequest request = exchange.getRequest();
        ServerHttpResponse response = exchange.getResponse();

        log.info("Custom Pre Filter: request id -> {}", request.getId());

        // Custom Post Filter
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            log.info("Custom Post Filter: response status code -> {}", response.getStatusCode());
        }));
    };
}

public static class Config {
    // Configuration properties for the filter can be added here
}
}

```

application.yml 파일 내 Custom Filter 등록

```

server:
  port: 8000

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://localhost:8761/eureka

spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: first-service
              uri: http://localhost:8081/
              predicates:
                - Path=/first-service/**
              filters:
                # - AddRequestHeader=f-request, 1st-request-header-by-yaml
                # - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
              uri: http://localhost:8082/
              predicates:

```

```

- Path=/second-service/**
filters:
# - AddRequestHeader=s-request, 2nd-request-header-by-yaml
# - AddResponseHeader=s-response, 2nd-response-header-from-yaml
- CustomFilter

```

Global Filter

애플리케이션 내의 **모든 또는 대부분의 요청**에 대해 공통적으로 적용되어 동작하는 필터

보통 프레임워크가 제공하는 메커니즘을 통해 애플리케이션의 전역(Global) 범위에 등록되어, 특정 조건 없이 들어오는 모든 요청 또는 특정 경로 그룹의 모든 요청에 자동으로 적용되는 필터

- 모든 요청에 대한 HTTPS 리다이렉트 처리
- 애플리케이션 전반에 걸친 보안 헤더(Ex. CORS, XSS 방어) 일괄 적용
- 모든 API 호출 시간을 측정하고 기록하는 성능 모니터링

Global Filter 정의

```

@Slf4j
@Component
public class GlobalFilter extends AbstractGatewayFilterFactory<GlobalFilter.Config> {

    public GlobalFilter() {
        super(Config.class);
    }

    @Override
    public GatewayFilter apply(Config config) {
        // Custom Pre Filter
        return (exchange, chain) -> {
            ServerHttpRequest request = exchange.getRequest();
            ServerHttpResponse response = exchange.getResponse();

            log.info("Global Filter baseMessage: {}", config.getBaseMessage());

            if (config.isPreLogger()) {
                log.info("Global Filter Start: request id -> {}", request.getId());
            }

            // Custom Post Filter
            return chain.filter(exchange).then(Mono.fromRunnable(() -> {
                if (config.isPostLogger()) {
                    log.info("Global Filter End: response status code -> {}", response.getStatusCode());
                }
            }));
        };
    }

    @Data
    public static class Config {
        private String baseMessage;
    }
}

```

```

    private boolean preLogger;
    private boolean postLogger;
  }
}

```

application.yml 파일 내 default-filters 추가

```

server:
  port: 8000

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://localhost:8761/eureka

spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
              args:
                baseMessage: Spring Cloud Gateway WebFlux Global Filter
                preLogger: true
                postLogger: true
          routes:
            - id: first-service
              uri: http://localhost:8081/
              predicates:
                - Path=/first-service/**
              filters:
                # - AddRequestHeader=f-request, 1st-request-header-by-yaml
                # - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
              uri: http://localhost:8082/
              predicates:
                - Path=/second-service/**
              filters:
                # - AddRequestHeader=s-request, 2nd-request-header-by-yaml
                # - AddResponseHeader=s-response, 2nd-response-header-from-yaml
                - CustomFilter

```

Logging Filter

들어오는 요청(Request)과 나가는 응답(Response)에 대한 **다양한 정보를 기록**(로깅)하는 역할을 하는 필터

요청이 들어오거나 응답이 나갈 때, 해당 요청의 경로, HTTP 메서드, 파라미터, 응답 상태 코드, 처리 시간 등 진단 및 모니터링에 필요한 정보를 수집하고 로그 파일에 기록하는 데 특화된 필터

Ex)

- 들어오는 모든 요청의 IP 주소, User-Agent, 요청 URL 기록
- 응답이 완료된 후 HTTP 상태 코드 및 응답 본문의 일부 기록
- 특정 작업(예: 회원가입, 결제)의 성공/실패 여부를 상세하게 로깅

Logging Filter 정의

```
@Slf4j
@Component
public class LoggingFilter extends AbstractGatewayFilterFactory<LoggingFilter.Config> {

    public LoggingFilter() {
        super(Config.class);
    }

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            ServerHttpRequest request = exchange.getRequest();
            ServerHttpResponse response = exchange.getResponse();

            log.info("Logging Filter baseMessage: {}, {}", config.getBaseMessage(), request.getRemoteAddress());

            if (config.isPreLogger()) {
                log.info("Logging Pre Filter: request uri = {}", request.getURI().toString());
            }
            return chain.filter(exchange).then(Mono.fromRunnable(() -> {
                if (config.isPostLogger()) {
                    log.info("Logging Post Filter: response status code = {}", response.getStatusCode());
                }
            }));
        };
    }

    @Data
    public static class Config {
        private String baseMessage;
        private boolean preLogger;
        private boolean postLogger;
    }
}
```

application.yml 파일 내 Logging Filter 등록

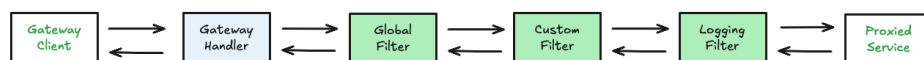
```
server:
  port: 8000
```

```

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://localhost:8761/eureka

spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
            args:
              baseMessage: Spring Cloud Gateway WebFlux Global Filter
              preLogger: true
              postLogger: true
          routes:
            - id: first-service
              uri: http://localhost:8081/
              predicates:
                - Path=/first-service/**
              filters:
                # - AddRequestHeader=f-request, 1st-request-header-by-yaml
                # - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
              uri: http://localhost:8082/
              predicates:
                - Path=/second-service/**
              filters:
                # - AddRequestHeader=s-request, 2nd-request-header-by-yaml
                # - AddResponseHeader=s-response, 2nd-response-header-from-yaml
                - name: CustomFilter
                - name: LoggingFilter
              args:
                baseMessage: Hi, there.
                preLogger: true
                postLogger: true

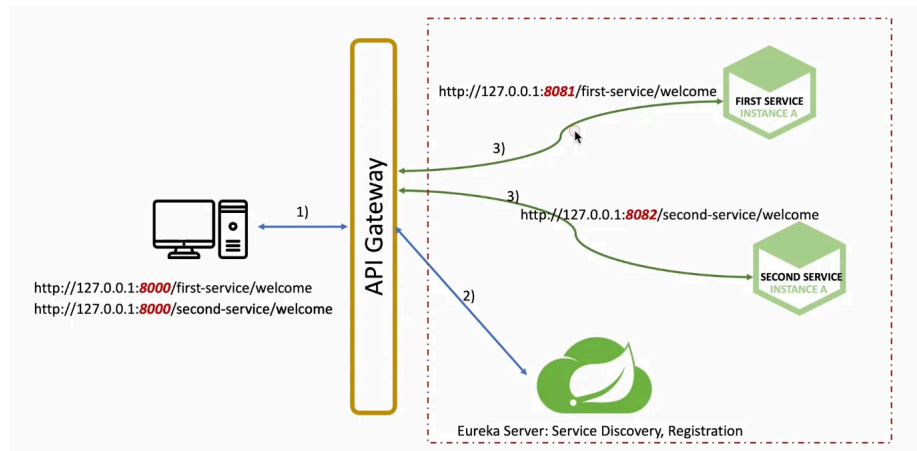
```



실행 순서

Spring Cloud Gateway - Eureka 연동

Load Balancer



1. `apigateway-service`, `first-service`, `second-service` Eureka 속성 파일 업데이트

```
eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
      defaultZone: http://localhost:8761/eureka
```

2. `apigateway-service` routes 속성 파일 업데이트

```
spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
              args:
                baseMessage: Spring Cloud Gateway WebFlux Global Filter
                preLogger: true
                postLogger: true
          routes:
            - id: first-service
              uri: lb://MY-FIRST-SERVICE
              predicates:
                - Path=/first-service/**
              filters:
                # - AddRequestHeader=f-request, 1st-request-header-by-yaml
                # - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
```

```

- id: second-service
  uri: lb://MY-SECOND-SERVICE
  predicates:
    - Path=/second-service/**
  filters:
#    - AddRequestHeader=s-request, 2nd-request-header-by-yaml
#    - AddResponseHeader=s-response, 2nd-response-header-from-yaml
    - name: CustomFilter
    - name: LoggingFilter
    args:
      baseMessage: Hi, there.
      preLogger: true
      postLogger: true

```

3. Eureka 서버 실행 (포트번호 8761)

4. `apigateway-service` , `first-service` , `second-service` 프로젝트 서버 실행

localhost:8761 실행 후 가동된 인스턴스 리스트 확인

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
APIGATEWAY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.25.43:apigateway-service:8000
MY-FIRST-SERVICE	n/a (1)	(1)	UP (1) - 192.168.25.43:725c3f3fbef84e5dc38820c2db83f5f8
MY-SECOND-SERVICE	n/a (1)	(1)	UP (1) - 192.168.25.43:2eac061781036f6a962e5a1f06935599